

due date. Form to be submitted via the 'Create Task' button, which will add the task to localStorage.

Line 4: Compared to older HTML text fields, you find three new attributes: *placeholder*, *autofocus* and *required*. The first attribute displays a hint to the user, in grey text, about the expected purpose/format of the input. The *autofocus* attribute places the cursor in the text field before and after form submission. The *required* attribute makes input mandatory before the form can be submitted.

Line 5-9: Task priorities drop-down; 'High', 'Normal' (default) and 'Low' entries.

Line 10: Text field for due date; format suggested via *placeholder*.

Line 11: Form submission button 'Create Task'.

Line 13: Space to list existing tasks. If no tasks are found, "Nothing much to do!" is displayed.

Operations and attributes of localStorage

There are three functions that perform operations on localStorage:

- **addTask():** Called when the user enters task details and clicks Create Task.
- **getTasks():** Retrieves all records from localStorage; called when the HTML page is loaded, and when a task is added, to update the list.
- **deleteTask():** Called when the user marks the task complete (ticks the check-box). The task is removed from localStorage.

```
1: function addTask()
2: {
3:   var values = new Array();
4:   var newDate = new Date();
5:   var itemId = newDate.getTime();
6:   var dueDate = "";
7:   if (!document.forms["todoForm"]["dueDate"].value)
8:   {
9:     dueDate = fillDueDate();
10:  }
11:  else
12:  {
13:    dueDate = document.forms["todoForm"]["dueDate"].value;
14:  }
15:  values.push(document.forms["todoForm"]["task"].value);
16:  values.push(document.forms["todoForm"]["priority"].value);
17:  values.push(dueDate);
18:  try {
19:    localStorage.setItem(itemId, values.join(';'));
20:  } catch (e) {
21:    if (e == QUOTA_EXCEEDED_ERR) {
22:      alert('Quota exceeded!');
23:    }
24:  }
25:  getTasks();

```

26: }

Line 3: Each record is stored as a key-value pair; fields are separated by a semicolon (;). A *values()* array is used to store each task record.

Line 4-5: Key is generated using the current time in milliseconds, on the assumption that there will always be at least a few milliseconds delay between the creation of two tasks—so, this is always unique.

Line 7-14: Check if the due date is entered; if not, insert today's date in *dd/mm/yyyy* format using the function *fillDueDate()*. If the user entered a due date, use it as-is, without validation, since our focus is on trying localStorage.

Line 18-24: Insert the new record in the localStorage database. If the 5 MB quota allotted by the browser is exceeded, an exception is thrown. Better storage management ideas are needed!

Line 25: Fetch tasks already in localStorage and display, including the newly created one.

```
1: function getTasks()
2: {
3:   var i = 0;
4:   var currentTaskInHTML = "";
5:   var allTasksInHTML = "";
6:   if (localStorage.length == 0)
7:   {
8:     document.getElementById('theTaskList').innerHTML =
"Nothing much to do!";
9:   }
10:  else
11:  {
12:    for (i=0; i<localStorage.length; i++)
13:    {
14:      var itemKey = localStorage.key(i);
15:      var values = localStorage.getItem(itemKey);
16:      values = values.split(";");
17:      currentTaskInHTML = '<input type="checkbox" '
name="task" value="\" + itemKey
+ \"\"
onClicK="deleteTask(' + itemKey +
')\"/> ' +
values[0]+ ' (' +values[1]+ ') ['
+values[2]+
']</BR>';
18:      allTasksInHTML += currentTaskInHTML;
19:    }
20:    document.getElementById('theTaskList').innerHTML =
allTasksInHTML;
21:  }
22: }

```

Line 6: If there are no tasks in localStorage, it displays "Nothing much to do!".